

```

        //

        return (BackgroundTaskRegistration ^)(cur);
    }

    hascur = iter->MoveNext();
}

//
// Register the background task.
//

auto builder = ref new BackgroundTaskBuilder();

builder->Name = name;
builder->TaskEntryPoint = taskEntryPoint;
builder->SetTrigger(trigger);

if (condition != nullptr) {

    builder->AddCondition(condition);
}

BackgroundTaskRegistration ^ task = builder->Register();

return task;
}

```

Related topics

- [Create and register an out-of-process background task](#)
- [Create and register an in-process background task](#)
- [Declare background tasks in the application manifest](#)
- [Handle a cancelled background task](#)
- [Monitor background task progress and completion](#)
- [Respond to system events with background tasks](#)
- [Set conditions for running a background task](#)
- [Update a live tile from a background task](#)
- [Use a maintenance trigger](#)
- [Run a background task on a timer](#)
- [Guidelines for background tasks](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Respond to system events with background tasks

Article • 10/20/2022

Important APIs

- [IBackgroundTask](#)
- [BackgroundTaskBuilder](#)
- [SystemTrigger](#)

Learn how to create a background task that responds to [SystemTrigger](#) events.

This topic assumes that you have a background task class written for your app, and that this task needs to run in response to an event triggered by the system such as when internet availability changes or the user logging in. This topic focuses on the [SystemTrigger](#) class. More information on writing a background task class is available in [Create and register an in-process background task](#) or [Create and register an out-of-process background task](#).

Create a SystemTrigger object

In your app code, create a new [SystemTrigger](#) object. The first parameter, *triggerType*, specifies the type of system event trigger that will activate this background task. For a list of event types, see [SystemTriggerType](#).

The second parameter, *OneShot*, specifies whether the background task will run only once the next time the system event occurs or every time the system event occurs until the task is unregistered.

The following code specifies that the background task runs whenever the Internet becomes available:

C#

```
SystemTrigger internetTrigger = new  
SystemTrigger(SystemTriggerType.InternetAvailable, false);
```

Register the background task

Register the background task by calling your background task registration function. For more information on registering background tasks, see [Register a background task](#).

The following code registers the background task for a background process that runs out-of-process. If you were calling a background task that runs in the same process as the host app, you would not set `entrypoint`:

C#

```
string entryPoint = "Tasks.ExampleBackgroundTaskClass"; // Namespace name,
'.', and the name of the class containing the background task
string taskName   = "Internet-based background task";

BackgroundTaskRegistration task = RegisterBackgroundTask(entryPoint,
taskName, internetTrigger, exampleCondition);
```

ⓘ Note

Universal Windows Platform apps must call **RequestAccessAsync** before registering any of the background trigger types.

To ensure that your Universal Windows app continues to run properly after you release an update, you must call **RemoveAccess** and then call **RequestAccessAsync** when your app launches after being updated. For more information, see [Guidelines for background tasks](#).

ⓘ Note

Background task registration parameters are validated at the time of registration. An error is returned if any of the registration parameters are invalid. Ensure that your app gracefully handles scenarios where background task registration fails - if instead your app depends on having a valid registration object after attempting to register a task, it may crash.

Remarks

To see background task registration in action, download the [background task sample](#) .

Background tasks can run in response to **SystemTrigger** and **MaintenanceTrigger** events, but you still need to [Declare background tasks in the application manifest](#). You must also call **RequestAccessAsync** before registering any background task type.

Apps can register background tasks that respond to [TimeTrigger](#), [PushNotificationTrigger](#), and [NetworkOperatorNotificationTrigger](#) events, enabling them to provide real-time communication with the user even when the app is not in the foreground. For more information, see [Support your app with background tasks](#).

Related topics

- [Create and register an out-of-process background task](#)
- [Create and register an in-process background task](#)
- [Declare background tasks in the application manifest](#)
- [Handle a cancelled background task](#)
- [Monitor background task progress and completion](#)
- [Register a background task](#)
- [Set conditions for running a background task](#)
- [Update a live tile from a background task](#)
- [Use a maintenance trigger](#)
- [Run a background task on a timer](#)
- [Guidelines for background tasks](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Run a background task on a timer

Article • 10/20/2022

Learn how to use the [TimeTrigger](#) to schedule a one-time background task, or run a periodic background task.

See **Scenario4** in the [Background activation sample](#) [↗](#) to see an example of how to implement the time triggered background task described in this topic.

This topic assumes that you have a background task that needs to run periodically, or at a specific time. If you don't already have a background task, there is a sample background task at [BackgroundActivity.cs](#) [↗](#). Or, follow the steps in [Create and register an in-process background task](#) or [Create and register an out-of-process background task](#) to create one.

Create a time trigger

Create a new [TimeTrigger](#). The second parameter, *OneShot*, specifies whether the background task will run only once or keep running periodically. If *OneShot* is set to true, the first parameter (*FreshnessTime*) specifies the number of minutes to wait before scheduling the background task. If *OneShot* is set to false, *FreshnessTime* specifies the frequency at which the background task will run.

The built-in timer for Universal Windows Platform (UWP) apps that target the desktop or mobile device family runs background tasks in 15-minute intervals. (The timer runs in 15-minute intervals so that the system only needs to wake once every 15 minutes to wake up the apps that have requested TimerTriggers--which saves power.)

- If *FreshnessTime* is set to 15 minutes and *OneShot* is true, the task will be scheduled to run once starting between 15 and 30 minutes from the time it is registered. If it is set to 25 minutes and *OneShot* is true, the task will be scheduled to run once starting between 25 and 40 minutes from the time it is registered.
- If *FreshnessTime* is set to 15 minutes and *OneShot* is false, the task will be scheduled to run every 15 minutes starting between 15 and 30 minutes from the time it is registered. If it is set to n minutes and *OneShot* is false, the task will be scheduled to run every n minutes starting between n and n + 15 minutes after it is registered.

 **Note**

If *FreshnessTime* is set to less than 15 minutes, an exception is thrown when attempting to register the background task.

For example, this trigger will cause a background task to run once an hour.

C#

```
TimeTrigger hourlyTrigger = new TimeTrigger(60, false);
```

C++/WinRT

```
Windows::ApplicationModel::Background::TimeTrigger hourlyTrigger{ 60, false  
};
```

C++

```
TimeTrigger ^ hourlyTrigger = ref new TimeTrigger(60, false);
```

(Optional) Add a condition

You can create a background task condition to control when the task runs. A condition prevents the background task from running until the condition is met. For more information, see [Set conditions for running a background task](#).

In this example the condition is set to **UserPresent** so that, once triggered, the task only runs once the user is active. For a list of possible conditions, see [SystemConditionType](#).

C#

```
SystemCondition userCondition = new  
SystemCondition(SystemConditionType.UserPresent);
```

C++/WinRT

```
Windows::ApplicationModel::Background::SystemCondition userCondition{  
    Windows::ApplicationModel::Background::SystemConditionType::UserPresent  
};
```

C++

```
SystemCondition ^ userCondition = ref new  
SystemCondition(SystemConditionType::UserPresent);
```

For more in-depth information on conditions and types of background triggers, see [Support your app with background tasks](#).

Call RequestAccessAsync()

Before registering the **ApplicationTrigger** background task, call **RequestAccessAsync** to determine the level of background activity the user allows because the user may have disabled background activity for your app. See [Optimize background activity](#) for more information about the ways users can control the settings for background activity.

C#

```
var requestStatus = await
Windows.ApplicationModel.Background.BackgroundExecutionManager.RequestAccess
Async();
if (requestStatus != BackgroundAccessStatus.AlwaysAllowed)
{
    // Depending on the value of requestStatus, provide an appropriate
    response
    // such as notifying the user which functionality won't work as expected
}
```

Register the background task

Register the background task by calling your background task registration function. For more information on registering background tasks, and to see the definition of the **RegisterBackgroundTask()** method in the sample code below, see [Register a background task](#).

Important

For background tasks that run in the same process as your app, do not set `entryPoint`. For background tasks that run in a separate process from your app, set `entryPoint` to be the namespace, '.', and the name of the class that contains your background task implementation.

C#

```
string entryPoint = "Tasks.ExampleBackgroundTaskClass";
string taskName   = "Example hourly background task";

BackgroundTaskRegistration task = RegisterBackgroundTask(entryPoint,
taskName, hourlyTrigger, userCondition);
```

C++/WinRT

```
std::wstring entryPoint{ L"Tasks.ExampleBackgroundTaskClass" };  
std::wstring taskName{ L"Example hourly background task" };  
  
Windows::ApplicationModel::Background::BackgroundTaskRegistration task{  
    RegisterBackgroundTask(entryPoint, taskName, hourlyTrigger,  
    userCondition) };
```

C++

```
String ^ entryPoint = "Tasks.ExampleBackgroundTaskClass";  
String ^ taskName   = "Example hourly background task";  
  
BackgroundTaskRegistration ^ task = RegisterBackgroundTask(entryPoint,  
taskName, hourlyTrigger, userCondition);
```

Background task registration parameters are validated at the time of registration. An error is returned if any of the registration parameters are invalid. Ensure that your app gracefully handles scenarios where background task registration fails - if instead your app depends on having a valid registration object after attempting to register a task, it may crash.

Manage resources for your background task

Use [BackgroundExecutionManager.RequestAccessAsync](#) to determine if the user has decided that your app's background activity should be limited. Be aware of your battery usage and only run in the background when it is necessary to complete an action that the user wants. See [Optimize background activity](#) for more information about the ways users can control the settings for background activity.

- **Memory:** Tuning your app's memory and energy use is key to ensuring that the operating system will allow your background task to run. Use the [Memory Management APIs](#) to see how much memory your background task is using. The more memory your background task uses, the harder it is for the OS to keep it running when another app is in the foreground. The user is ultimately in control of all background activity that your app can perform and has visibility on the impact your app has on battery use.
- **CPU time:** Background tasks are limited by the amount of wall-clock usage time they get based on trigger type.

See [Support your app with background tasks](#) for the resource constraints applied to background tasks.

Remarks

Starting with Windows 10, it is no longer necessary for the user to add your app to the lock screen in order to utilize background tasks.

A background task will only run using a **TimeTrigger** if you have called [RequestAccessAsync](#) first.

Related topics

- [Guidelines for background tasks](#)
- [Background task code sample](#) 
- [Create and register an in-process background task](#)
- [Create and register an out-of-process background task](#)
- [Debug a background task](#)
- [Declare background tasks in the application manifest](#)
- [Free memory when your app moves to the background](#)
- [Handle a cancelled background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)
- [Monitor background task progress and completion](#)
- [Postpone app suspension with extended execution](#)
- [Register a background task](#)
- [Respond to system events with background tasks](#)
- [Set conditions for running a background task](#)
- [Update a live tile from a background task](#)
- [Use a maintenance trigger](#)

Run a background task when your UWP app is updated

Article • 10/20/2022

Learn how to write a background task that runs after your Universal Windows Platform (UWP) store app is updated.

The Update Task background task is invoked by the operating system after the user installs an update to an app that is installed on the device. This allows your app to perform initialization tasks such as initializing a new push notification channel, updating database schema, and so on, before the user launches your updated app.

The Update Task differs from launching a background task using the [ServicingComplete](#) trigger because in that case your app must run at least once before it is updated in order to register the background task that will be activated by the **ServicingComplete** trigger. The Update Task isn't registered and so an app that has never been run, but that is upgraded, will still have its update task triggered.

Step 1: Create the background task class

As with other types of background tasks, you implement the Update Task background task as a Windows Runtime component. To create this component, follow the steps in the **Create the Background Task class** section of [Create and register an out-of-process background task](#). The steps include:

- Adding a Windows Runtime component project to your solution.
- Creating a reference from your app to the component.
- Creating a public, sealed class in the component that implements [IBackgroundTask](#).
- Implementing the [Run](#) method, which is the required entry point that is called when the Update Task is run. If you are going to make asynchronous calls from your background task, [Create and register an out-of-process background task](#) explains how to use a deferral in your **Run** method.

You don't need to register this background task (the "Register the background task to run" section in the **Create and register an out-of-process background task** topic) to use the Update Task. This is the main reason to use an Update Task because you don't need to add any code to your app to register the task and the app doesn't have to at least run once before being updated to register the background task.

The following sample code shows a basic starting point for an Update Task background task class in C#. The background task class itself - and all other classes in the background task project - need to be **public** and **sealed**. Your background task class must derive from **IBackgroundTask** and have a public **Run()** method with the signature shown below:

CS

```
using Windows.ApplicationModel.Background;

namespace BackgroundTasks
{
    public sealed class UpdateTask : IBackgroundTask
    {
        public void Run(IBackgroundTaskInstance taskInstance)
        {
            // your app migration/update code here
        }
    }
}
```

Step 2: Declare your background task in the package manifest

In the Visual Studio Solution Explorer, right-click **Package.appxmanifest** and click **View Code** to view the package manifest. Add the following **<Extensions>** XML to declare your update task:

XML

```
<Package ...>
    ...
    <Applications>
        <Application ...>
            ...
            <Extensions>
                <Extension Category="windows.updateTask"
EntryPoint="BackgroundTasks.UpdateTask">
                </Extension>
            </Extensions>

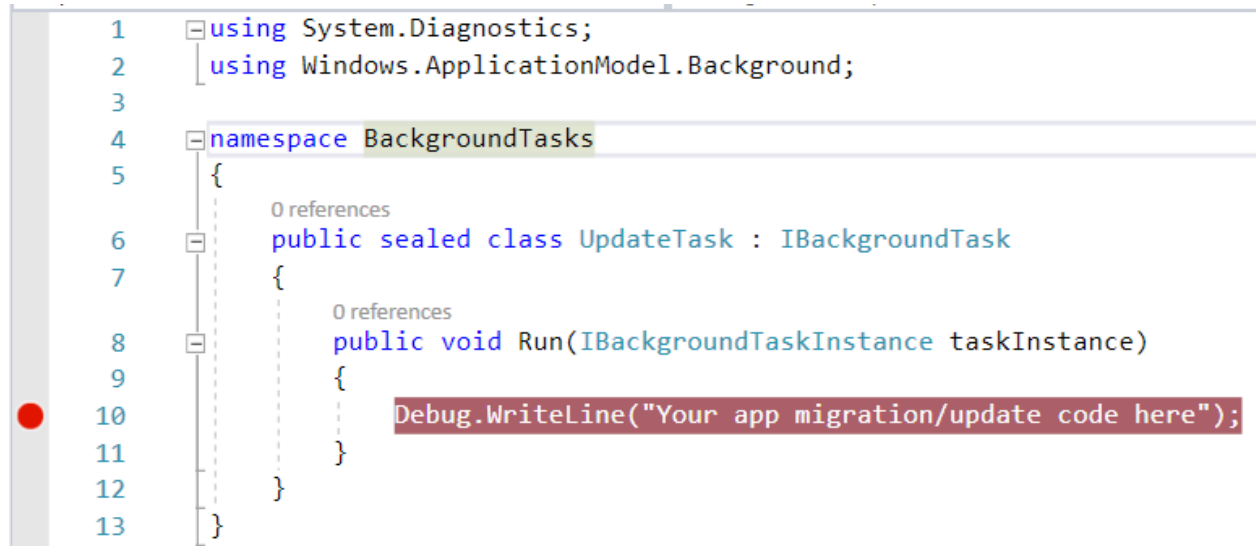
        </Application>
    </Applications>
</Package>
```

In the XML above, ensure that the `EntryPoint` attribute is set to the namespace.class name of your update task class. The name is case-sensitive.

Step 3: Debug/test your Update task

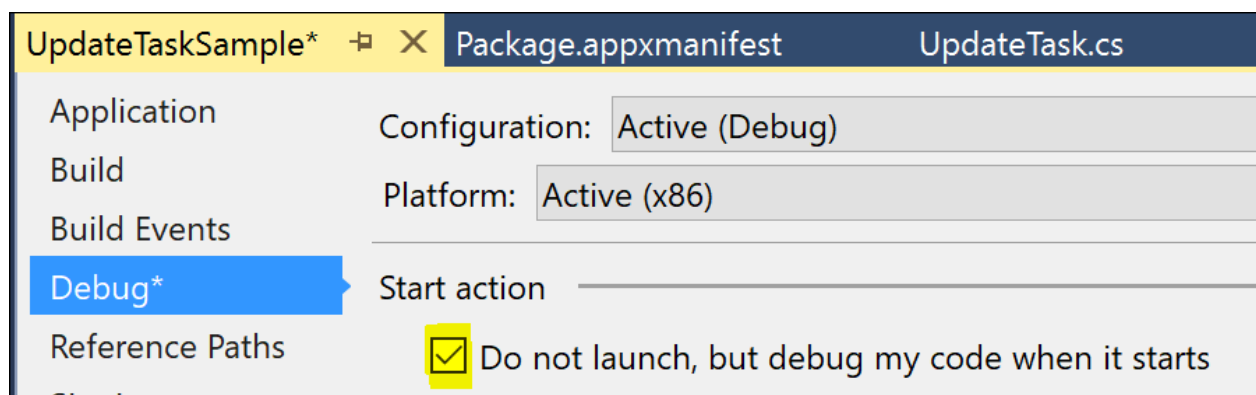
Ensure that you have deployed your app to your machine so that there is something to update.

Set a breakpoint in the `Run()` method of your background task.



```
1  using System.Diagnostics;
2  using Windows.ApplicationModel.Background;
3
4  namespace BackgroundTasks
5  {
6      0 references
7      public sealed class UpdateTask : IBackgroundTask
8      {
9          0 references
10         public void Run(IBackgroundTaskInstance taskInstance)
11         {
12             Debug.WriteLine("Your app migration/update code here");
13         }
14     }
15 }
```

Next, in the solution explorer, right-click your app's project (not the background task project) and then click **Properties**. In the application Properties window, click **Debug** on the left, then select **Do not launch, but debug my code when it starts**:



Next, to ensure that the `UpdateTask` is triggered, increase the package's version number. In the Solution Explorer, double-click your app's **Package.appxmanifest** file to open the package designer, and then update the **Build** number:

Package.appxmanifest

Application Visual Assets Capabilities Declarations Content URIs **Packaging**

Use this page to set the properties that identify and describe your package when it is deployed.

Package name: 532bae71-8e4b-4d1e-b082-64f499665df0

Package display name: UpdateTaskSample

Version: Major: 1 Minor: 0 Build: 1 [More information](#)

Now, in Visual Studio 2019 when you press F5, your app will be updated and the system will activate your UpdateTask component in the background. The debugger will automatically attach to the background process. Your breakpoint will get hit and you can step through your update code logic.

When the background task completes, you can launch the foreground app from the Windows start menu within the same debug session. The debugger will again automatically attach, this time to your foreground process, and you can step through your app's logic.

ⓘ Note

Visual Studio 2015 users: The above steps apply to Visual Studio 2017 or Visual Studio 2019. If you are using Visual Studio 2015, you can use the same techniques to trigger and test the UpdateTask, except Visual Studio will not attach to it. An alternative procedure in VS 2015 is to setup an [ApplicationTrigger](#) that sets the UpdateTask as its Entry Point, and trigger the execution directly from the foreground app.

See also

[Create and register an out-of-process background task](#)

Feedback

Was this page helpful?

Yes

No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Run in the background indefinitely

Article • 10/20/2022

To provide the best experience for users, Windows imposes resource limits on Universal Windows Platform (UWP) apps. Foreground apps are given the most memory and execution time; background apps get less. Users are thus protected from poor foreground app performance and heavy battery drain.

However, developers writing UWP apps for personal use (that is, side loaded apps that won't be published in the Microsoft Store), or developers writing Enterprise UWP apps, may want to use all resources available on the device without any background or extended execution throttling. Line of business and personal UWP applications can use APIs in the Windows Creators Update (version 1703) to turn off throttling. Be aware that you can't put an app into the Microsoft Store if it uses these APIs.

Run while minimized

UWP apps move to a suspended state when they are not running in the foreground. On desktop, this occurs when a user minimizes the app. Apps use an extended execution session in order to continue running while minimized. The extended execution APIs that are accepted by the Microsoft Store are detailed in [Postpone app suspension with extended execution](#).

If you are developing an app that is not intended to be submitted into the Microsoft Store, then you can use the `ExtendedExecutionForegroundSession` with the `extendedExecutionUnconstrained` restricted capability so that your app can continue to run while minimized, regardless of the energy state of the device.

The `extendedExecutionUnconstrained` capability is added as a restricted capability in your app's manifest. See [App capability declarations](#) for more information about restricted capabilities.

ⓘ Note

Add the `xmlns:rescap` XML namespace declaration, and use the `rescap` prefix to declare the capability.

For more information, see the Restricted Capabilities section of [App capability declarations](#).

XML

```
<Package
  ...

  xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows1
  0/restrictedcapabilities"
  IgnorableNamespaces="uap mp rescap">
  ...
  <Capabilities>
    <rescap:Capability Name="extendedExecutionUnconstrained"/>
  </Capabilities>
</Package>
```

When you use the `extendedExecutionUnconstrained` capability, `ExtendedExecutionForegroundSession` and `ExtendedExecutionForegroundReason` are used rather than `ExtendedExecutionSession` and `ExtendedExecutionReason`. The same pattern for creating the session, setting members, and requesting the extension asynchronously still applies:

C#

```
var newSession = new ExtendedExecutionForegroundSession();
newSession.Reason = ExtendedExecutionForegroundReason.Unconstrained;
newSession.Description = "Long Running Processing";
newSession.Revoked += SessionRevoked;
ExtendedExecutionForegroundResult result = await
newSession.RequestExtensionAsync();
switch (result)
{
    case ExtendedExecutionForegroundResult.Allowed:
        DoLongRunningWork();
        break;

    default:
        case ExtendedExecutionForegroundResult.Denied:
            DoShortRunningWork();
            break;
}
```

You can request this extended execution session as soon as the app comes to the foreground. Unconstrained extended execution sessions are not limited by energy quotas or by the operating system battery saver. As long as a reference to the session object exists, the app will stay in the running state and not enter the suspended state. If the app is closed by the user, the session will be revoked.

Registering for the **Revoked** event will enable your app to do any cleanup work required. In the suspending state, you can create an extended execution session with [ExtendedExecutionReason.SavingData](#) to save user data before the app is terminated and removed from memory.

Run background tasks indefinitely

In the Universal Windows Platform, background tasks are processes that run in the background without any form of user interface. Background tasks may generally run for a maximum of twenty-five seconds before they are cancelled. Some of the longer-running tasks also have a check to ensure that the background task is not sitting idle or using memory. In the Windows Creators Update (version 1703), the [extendedBackgroundTaskTime](#) restricted capability was introduced to remove these limits. The **extendedBackgroundTaskTime** capability is added as a restricted capability in your app's manifest file:

ⓘ Note

Add the *xmlns:rescap* XML namespace declaration, and use the *rescap* prefix to declare the capability.

For more information, see the Restricted Capabilities section of [App capability declarations](#).

Package.appxmanifest

XML

```
<Package
  ...

  xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows10/restrictedcapabilities"
  IgnorableNamespaces="uap mp rescap">
  ...
  <Capabilities>
    <rescap:Capability Name="extendedBackgroundTaskTime"/>
  </Capabilities>
</Package>
```

This capability removes execution time limitations and the idle task watchdog. Once a background task has started, whether by a trigger or an app service call, once it takes a deferral on the [BackgroundTaskInstance](#) provided by the **Run** method, it can run

indefinitely. If the app is set to **Managed By Windows**, then it still may have an energy quota applied to it, and its background tasks will not be activated when Battery Saver is active. This can be changed with OS settings. More information is available in [Optimizing Background Activity](#).

The Universal Windows Platform monitors background task execution to ensure good battery life and a smooth foreground app experience. However, personal apps and Enterprise line-of-Business apps can use extended execution and the **extendedBackgroundTaskTime** capability to create apps that will run as long as needed regardless of the device's resource availability.

Be aware that the **extendedExecutionUnconstrained** and **extendedBackgroundTaskTime** capabilities can override default policy for UWP apps and may cause significant battery drain. Before using these capabilities, first confirm that the default extended execution and background task time policies do not meet your needs and perform testing in battery-constrained conditions to understand the impact your app will have on a device.

See also

[Remove background task resource restrictions](#)

Set conditions for running a background task

Article • 10/20/2022

Important APIs

- [SystemCondition](#)
- [SystemConditionType](#)
- [BackgroundTaskBuilder](#)

Learn how to set conditions that control when your background task will run.

Sometimes, background tasks require certain conditions to be met for the background task to succeed. You can specify one or more of the conditions specified by [SystemConditionType](#) when registering your background task. The condition will be checked after the trigger has been fired. The background task will then be queued, but it will not run until all the required conditions are satisfied.

Putting conditions on background tasks saves battery life and CPU by preventing tasks from running unnecessarily. For example, if your background task runs on a timer and requires Internet connectivity, add the **InternetAvailable** condition to the [TaskBuilder](#) before registering the task. This will help prevent the task from using system resources and battery life unnecessarily by only running the background task when the timer has elapsed *and* the Internet is available.

It is also possible to combine multiple conditions by calling **AddCondition** multiple times on the same [TaskBuilder](#). Take care not to add conflicting conditions, such as **UserPresent** and **UserNotPresent**.

Create a SystemCondition object

This topic assumes that you have a background task already associated with your app, and that your app already includes code that creates a [BackgroundTaskBuilder](#) object named **taskBuilder**. See [Create and register an in-process background task](#) or [Create and register an out-of-process background task](#) if you need to create a background task first.

This topic applies to background tasks that run out-of-process as well as those that run in the same process as the foreground app.

Before adding the condition, create a [SystemCondition](#) object to represent the condition that must be in effect for a background task to run. In the constructor, specify the condition that must be met with a [SystemConditionType](#) enumeration value.

The following code creates a [SystemCondition](#) object that specifies the **InternetAvailable** condition:

C#

```
SystemCondition internetCondition = new  
SystemCondition(SystemConditionType.InternetAvailable);
```

Add the SystemCondition object to your background task

To add the condition, call the [AddCondition](#) method on the [BackgroundTaskBuilder](#) object, and pass it the [SystemCondition](#) object.

The following code uses **taskBuilder** to add the **InternetAvailable** condition.

C#

```
taskBuilder.AddCondition(internetCondition);
```

Register your background task

Now you can register your background task with the [Register](#) method, and the background task will not start until the specified condition is met.

The following code registers the task and stores the resulting **BackgroundTaskRegistration** object:

C#

```
BackgroundTaskRegistration task = taskBuilder.Register();
```

ⓘ Note

Background task registration parameters are validated at the time of registration. An error is returned if any of the registration parameters are invalid. Ensure that your app gracefully handles scenarios where background task registration fails - if

instead your app depends on having a valid registration object after attempting to register a task, it may crash.

Place multiple conditions on your background task

To add multiple conditions, your app makes multiple calls to the [AddCondition](#) method. These calls must come before task registration to be effective.

ⓘ Note

Take care not to add conflicting conditions to a background task.

The following snippet shows multiple conditions in the context of creating and registering a background task.

C#

```
// Set up the background task.
TimeTrigger hourlyTrigger = new TimeTrigger(60, false);

var recurringTaskBuilder = new BackgroundTaskBuilder();

recurringTaskBuilder.Name           = "Hourly background task";
recurringTaskBuilder.TaskEntryPoint = "Tasks.ExampleBackgroundTaskClass";
recurringTaskBuilder.SetTrigger(hourlyTrigger);

// Begin adding conditions.
SystemCondition userCondition      = new
SystemCondition(SystemConditionType.UserPresent);
SystemCondition internetCondition = new
SystemCondition(SystemConditionType.InternetAvailable);

recurringTaskBuilder.AddCondition(userCondition);
recurringTaskBuilder.AddCondition(internetCondition);

// Done adding conditions, now register the background task.

BackgroundTaskRegistration task = recurringTaskBuilder.Register();
```

Remarks

ⓘ Note

Choose conditions for your background task so that it only runs when it's needed, and doesn't run when it shouldn't. See **SystemConditionType** for descriptions of the different background task conditions.

Related topics

- [Create and register an out-of-process background task](#)
- [Create and register an in-process background task](#)
- [Declare background tasks in the application manifest](#)
- [Handle a cancelled background task](#)
- [Monitor background task progress and completion](#)
- [Register a background task](#)
- [Respond to system events with background tasks](#)
- [Update a live tile from a background task](#)
- [Use a maintenance trigger](#)
- [Run a background task on a timer](#)
- [Guidelines for background tasks](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Trigger a background task from within your app

Article • 10/20/2022

Learn how to use the [ApplicationTrigger](#) to activate a background task from within your app.

For an example of how to create an Application trigger, see this [example](#).

This topic assumes that you have a background task that you want to activate from your application. If you don't already have a background task, there is a sample background task at [BackgroundActivity.cs](#). Or, follow the steps in [Create and register an out-of-process background task](#) to create one.

Why use an application trigger

Use an **ApplicationTrigger** to run code in a separate process from the foreground app. An **ApplicationTrigger** is appropriate if your app has work that needs to be done in the background--even if the user closes the foreground app. If background work should halt when the app is closed, or should be tied to the state of the foreground process, then [Extended Execution](#) should be used, instead.

Create an application trigger

Create a new [ApplicationTrigger](#). You could store it in a field as is done in the snippet below. This is for convenience so that we don't have to create a new instance later when we want to signal the trigger. But you can use any **ApplicationTrigger** instance to signal the trigger.

C#

```
// _AppTrigger is an ApplicationTrigger field defined at a scope that will
// keep it alive
// as long as you need to trigger the background task.
// Or, you could create a new ApplicationTrigger instance and use that when
// you want to
// trigger the background task.
_AppTrigger = new ApplicationTrigger();
```

C++/WinRT

```
// _AppTrigger is an ApplicationTrigger field defined at a scope that will
// keep it alive
// as long as you need to trigger the background task.
// Or, you could create a new ApplicationTrigger instance and use that when
// you want to
// trigger the background task.
Windows::ApplicationModel::Background::ApplicationTrigger _AppTrigger;
```

C++

```
// _AppTrigger is an ApplicationTrigger field defined at a scope that will
// keep it alive
// as long as you need to trigger the background task.
// Or, you could create a new ApplicationTrigger instance and use that when
// you want to
// trigger the background task.
ApplicationTrigger ^ _AppTrigger = ref new ApplicationTrigger();
```

(Optional) Add a condition

You can create a background task condition to control when the task runs. A condition prevents the background task from running until the condition is met. For more information, see [Set conditions for running a background task](#).

In this example the condition is set to **InternetAvailable** so that, once triggered, the task only runs once internet access is available. For a list of possible conditions, see [SystemConditionType](#).

C#

```
SystemCondition internetCondition = new
SystemCondition(SystemConditionType.InternetAvailable);
```

C++/WinRT

```
Windows::ApplicationModel::Background::SystemCondition internetCondition{

Windows::ApplicationModel::Background::SystemConditionType::InternetAvailabl
e };
```

C++

```
SystemCondition ^ internetCondition = ref new
SystemCondition(SystemConditionType::InternetAvailable)
```

For more in-depth information on conditions and types of background triggers, see [Support your app with background tasks](#).

Call RequestAccessAsync()

Before registering the **ApplicationTrigger** background task, call **RequestAccessAsync** to determine the level of background activity the user allows because the user may have disabled background activity for your app. See [Optimize background activity](#) for more information about the ways users can control the settings for background activity.

C#

```
var requestStatus = await
Windows.ApplicationModel.Background.BackgroundExecutionManager.RequestAccess
Async();
if (requestStatus != BackgroundAccessStatus.AlwaysAllowed)
{
    // Depending on the value of requestStatus, provide an appropriate
    response
    // such as notifying the user which functionality won't work as expected
}
```

Register the background task

Register the background task by calling your background task registration function. For more information on registering background tasks, and to see the definition of the **RegisterBackgroundTask()** method in the sample code below, see [Register a background task](#).

If you are considering using an Application Trigger to extend the lifetime of your foreground process, consider using [Extended Execution](#) instead. The Application Trigger is designed for creating a separately hosted process to do work in. The following code snippet registers an out-of-process background trigger.

C#

```
string entryPoint = "Tasks.ExampleBackgroundTaskClass";
string taskName   = "Example application trigger";

BackgroundTaskRegistration task = RegisterBackgroundTask(entryPoint,
taskName, appTrigger, internetCondition);
```

C++/WinRT

```
std::wstring entryPoint{ L"Tasks.ExampleBackgroundTaskClass" };
std::wstring taskName{ L"Example application trigger" };

Windows::ApplicationModel::Background::BackgroundTaskRegistration task{
    RegisterBackgroundTask(entryPoint, taskName, appTrigger,
        internetCondition) };

```

C++

```
String ^ entryPoint = "Tasks.ExampleBackgroundTaskClass";
String ^ taskName    = "Example application trigger";

BackgroundTaskRegistration ^ task = RegisterBackgroundTask(entryPoint,
    taskName, appTrigger, internetCondition);

```

Background task registration parameters are validated at the time of registration. An error is returned if any of the registration parameters are invalid. Ensure that your app gracefully handles scenarios where background task registration fails - if instead your app depends on having a valid registration object after attempting to register a task, it may crash.

Trigger the background task

Before you trigger the background task, use [BackgroundTaskRegistration](#) to verify that the background task is registered. A good time to verify that all of your background tasks are registered is during app launch.

Trigger the background task by calling [ApplicationTrigger.RequestAsync](#). Any **ApplicationTrigger** instance will do.

Note that **ApplicationTrigger.RequestAsync** can't be called from the background task itself, or when the app is in the background running state (see [App lifecycle](#) for more information about application states). It may return [DisabledByPolicy](#) if the user has set energy or privacy policies that prevent the app from performing background activity. Also, only one AppTrigger can run at a time. If you attempt to run an AppTrigger while another is already running, the function will return [CurrentlyRunning](#).

C#

```
var result = await _AppTrigger.RequestAsync();

```

Manage resources for your background task

Use [BackgroundExecutionManager.RequestAccessAsync](#) to determine if the user has decided that your app's background activity should be limited. Be aware of your battery usage and only run in the background when it is necessary to complete an action that the user wants. See [Optimize background activity](#) for more information about the ways users can control the settings for background activity.

- **Memory:** Tuning your app's memory and energy use is key to ensuring that the operating system will allow your background task to run. Use the [Memory Management APIs](#) to see how much memory your background task is using. The more memory your background task uses, the harder it is for the OS to keep it running when another app is in the foreground. The user is ultimately in control of all background activity that your app can perform and has visibility on the impact your app has on battery use.
- **CPU time:** Background tasks are limited by the amount of wall-clock usage time they get based on trigger type. Background tasks triggered by the Application trigger are limited to about 10 minutes.

See [Support your app with background tasks](#) for the resource constraints applied to background tasks.

Remarks

Starting with Windows 10, it is no longer necessary for the user to add your app to the lock screen in order to utilize background tasks.

A background task will only run using an **ApplicationTrigger** if you have called [RequestAccessAsync](#) first.

Related topics

- [Guidelines for background tasks](#)
- [Background task code sample](#) 
- [Create and register an in-process background task.](#)
- [Create and register an out-of-process background task](#)
- [Debug a background task](#)
- [Declare background tasks in the application manifest](#)
- [Free memory when your app moves to the background](#)
- [Handle a cancelled background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)
- [Monitor background task progress and completion](#)

- Postpone app suspension with extended execution
- Register a background task
- Respond to system events with background tasks
- Set conditions for running a background task
- Update a live tile from a background task
- Use a maintenance trigger

Update a live tile from a background task

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Use a background task to update your app's live tile with fresh content.

Important APIs

- [IBackgroundTask](#)
- [BackgroundTaskBuilder](#)

Create the background task project

To enable a live tile for your app, add a new Windows Runtime component project to your solution. This is a separate assembly that the OS loads and runs in the background when a user installs your app.

1. In Solution Explorer, right-click the solution, click **Add**, and then click **New Project**.
2. In the **Add New Project** dialog, select the **Windows Runtime Component** template in the **Installed > Other Languages > Visual C# > Windows Universal** section.
3. Name the project **BackgroundTasks** and click or tap **OK**. Microsoft Visual Studio adds the new project to the solution.
4. In the main project, add a reference to the **BackgroundTasks** project.

Implement the background task

Implement the [IBackgroundTask](#) interface to create a class that updates your app's live tile. Your background work goes in the **Run** method. In this case, the task gets a syndication feed for the MSDN blogs. To prevent the task from closing prematurely while asynchronous code is still running, get a deferral.

1. In Solution Explorer, rename the automatically generated file, **Class1.cs**, to **BlogFeedBackgroundTask.cs**.

2. In `BlogFeedBackgroundTask.cs`, replace the automatically generated code with the stub code for the **BlogFeedBackgroundTask** class.
3. In the `Run` method implementation, add code for the **GetMSDNBlogFeed** and **UpdateTile** methods.

CS

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

// Added during quickstart
using Windows.ApplicationModel.Background;
using Windows.Data.Xml.Dom;
using Windows.UI.Notifications;
using Windows.Web.Syndication;

namespace BackgroundTasks
{
    public sealed class BlogFeedBackgroundTask : IBackgroundTask
    {
        public async void Run( IBackgroundTaskInstance taskInstance )
        {
            // Get a deferral, to prevent the task from closing prematurely
            // while asynchronous code is still running.
            BackgroundTaskDeferral deferral = taskInstance.GetDeferral();

            // Download the feed.
            var feed = await GetMSDNBlogFeed();

            // Update the live tile with the feed items.
            UpdateTile( feed );

            // Inform the system that the task is finished.
            deferral.Complete();
        }

        private static async Task<SyndicationFeed> GetMSDNBlogFeed()
        {
            SyndicationFeed feed = null;

            try
            {
                // Create a syndication client that downloads the feed.
                SyndicationClient client = new SyndicationClient();
                client.BypassCacheOnRetrieve = true;
                client.SetRequestHeader( customHeaderName, customHeaderValue );

                // Download the feed.
            }
        }
    }
}
```

```

        feed = await client.RetrieveFeedAsync( new Uri( feedUrl ) );
    }
    catch( Exception ex )
    {
        Debug.WriteLine( ex.ToString() );
    }

    return feed;
}

private static void UpdateTile( SyndicationFeed feed )
{
    // Create a tile update manager for the specified syndication
feed.
    var updater =
TileUpdateManager.CreateTileUpdaterForApplication();
    updater.EnableNotificationQueue( true );
    updater.Clear();

    // Keep track of the number feed items that get tile
notifications.
    int itemCount = 0;

    // Create a tile notification for each feed item.
    foreach( var item in feed.Items )
    {
        XmlDocument tileXml = TileUpdateManager.GetTemplateContent(
TileTemplateType.TileWide310x150Text03 );

        var title = item.Title;
        string titleText = title.Text == null ? String.Empty :
title.Text;
        tileXml.GetElementsByTagName( textElementName )[0].InnerText
= titleText;

        // Create a new tile notification.
        updater.Update( new TileNotification( tileXml ) );

        // Don't create more than 5 notifications.
        if( itemCount++ > 5 ) break;
    }
}

// Although most HTTP servers do not require User-Agent header,
others will reject the request or return
// a different response if this header is missing. Use
SetRequestHeader() to add custom headers.
static string customHeaderName = "User-Agent";
static string customHeaderValue = "Mozilla/5.0 (compatible; MSIE
10.0; Windows NT 6.2; WOW64; Trident/6.0)";

static string textElementName = "text";
static string feedUrl = @"http://blogs.msdn.com/b/MainFeed.aspx?
Type=BlogsOnly";

```

```
}  
}
```

Set up the package manifest

To set up the package manifest, open it and add a new background task declaration. Set the entry point for the task to the class name, including its namespace.

1. In Solution Explorer, open `Package.appxmanifest`.
2. Click or tap the **Declarations** tab.
3. Under **Available Declarations**, select **BackgroundTasks** and click **Add**. Visual Studio adds **BackgroundTasks** under **Supported Declarations**.
4. Under **Supported task types**, ensure that **Timer** is checked.
5. Under **App settings**, set the entry point to **BackgroundTasks.BlogFeedBackgroundTask**.
6. Click or tap the **Application UI** tab.
7. Set **Lock screen notifications** to **Badge and Tile Text**.
8. Set a path to a 24x24 pixel icon in the **Badge logo** field. **Important** This icon must use monochrome and transparent pixels only.
9. In the **Small logo** field, set a path to a 30x30 pixel icon.
10. In the **Wide logo** field, set a path to a 310x150 pixel icon.

Register the background task

Create a [BackgroundTaskBuilder](#) to register your task.

Note Starting in Windows 8.1, background task registration parameters are validated at the time of registration. An error is returned if any of the registration parameters are invalid. Your app must be able to handle scenarios where background task registration fails - for example, use a conditional statement to check for registration errors and then retry failed registration using different parameter values.

In your app's main page, add the **RegisterBackgroundTask** method and call it in the **OnNavigatedTo** event handler.

CS

```
using System;  
using System.Collections.Generic;  
using System.IO;  
using System.Linq;
```

```

using System.Threading.Tasks;
using Windows.ApplicationModel.Background;
using Windows.Data.Xml.Dom;
using Windows.Foundation;
using Windows.Foundation.Collections;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Data;
using Windows.UI.Xaml.Input;
using Windows.UI.Xaml.Media;
using Windows.UI.Xaml.Navigation;
using Windows.Web.Syndication;

// The Blank Page item template is documented at
https://go.microsoft.com/fwlink/p/?LinkID=234238

namespace ContosoApp
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within a
    Frame.
    /// </summary>
    public sealed partial class MainPage : Page
    {
        public MainPage()
        {
            this.InitializeComponent();
        }

        /// <summary>
        /// Invoked when this page is about to be displayed in a Frame.
        /// </summary>
        /// <param name="e">Event data that describes how this page was
        reached. The Parameter
        /// property is typically used to configure the page.</param>
        protected override void OnNavigatedTo( NavigationEventArgs e )
        {
            this.RegisterBackgroundTask();
        }

        private async void RegisterBackgroundTask()
        {
            var backgroundAccessStatus = await
BackgroundExecutionManager.RequestAccessAsync();
            if( backgroundAccessStatus ==
BackgroundAccessStatus.AllowedSubjectToSystemPolicy ||
                backgroundAccessStatus ==
BackgroundAccessStatus.AlwaysAllowed )
            {
                foreach( var task in BackgroundTaskRegistration.AllTasks )
                {
                    if( task.Value.Name == taskName )
                    {

```

```

        task.Value.Unregister( true );
    }
}

BackgroundTaskBuilder taskBuilder = new
BackgroundTaskBuilder();
taskBuilder.Name = taskName;
taskBuilder.TaskEntryPoint = taskEntryPoint;
taskBuilder.SetTrigger( new TimeTrigger( 15, false ) );
var registration = taskBuilder.Register();
}
}

private const string taskName = "BlogFeedBackgroundTask";
private const string taskEntryPoint =
"BackgroundTasks.BlogFeedBackgroundTask";
}
}

```

Debug the background task

To debug the background task, set a breakpoint in the task's Run method. In the **Debug Location** toolbar, select your background task. This causes the system to call the Run method immediately.

1. Set a breakpoint in the task's Run method.
2. Press F5 or tap **Debug > Start Debugging** to deploy and run the app.
3. After the app launches, switch back to Visual Studio.
4. Ensure that the **Debug Location** toolbar is visible. It's on the **View > Toolbars** menu.
5. On the **Debug Location** toolbar, click the **Suspend** dropdown and select **BlogFeedBackgroundTask**.
6. Visual Studio suspends execution at the breakpoint.
7. Press F5 or tap **Debug > Continue** to continue running the app.
8. Press Shift+F5 or tap **Debug > Stop Debugging** to stop debugging.
9. Return to the app's tile on the Start screen. After a few seconds, tile notifications appear on your app's tile.

Related topics

- [BackgroundTaskBuilder](#)
- [TileUpdateManager](#)
- [TileNotification](#)
- [Support your app with background tasks](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Use a maintenance trigger

Article • 10/20/2022

Important APIs

- [MaintenanceTrigger](#)
- [BackgroundTaskBuilder](#)
- [SystemCondition](#)

Learn how to use the [MaintenanceTrigger](#) class to run lightweight code in the background while the device is plugged in.

Create a maintenance trigger object

This example assumes that you have lightweight code you can run in the background to enhance your app while the device is plugged in. This topic focuses on the [MaintenanceTrigger](#), which is similar to [SystemTrigger](#).

More information on writing a background task class is available in [Create and register an in-process background task](#) or [Create and register an out-of-process background task](#).

Create a new [MaintenanceTrigger](#) object. The second parameter, *OneShot*, specifies whether the maintenance task will run only once or continue to run periodically. If *OneShot* is set to true, the first parameter (*FreshnessTime*) specifies the number of minutes to wait before scheduling the background task. If *OneShot* is set to false, *FreshnessTime* specifies how often the background task will run.

ⓘ Note

If *FreshnessTime* is set to less than 15 minutes, an exception is thrown when attempting to register the background task.

This example code creates a trigger that runs once an hour.

C#

```
uint waitIntervalMinutes = 60;
MaintenanceTrigger taskTrigger = new MaintenanceTrigger(waitIntervalMinutes,
false);
```

(Optional) Add a condition

- If necessary, create a background task condition to control when the task runs. A condition prevents your background task from running until the condition is met - for more information, see [Set conditions for running a background task](#)

In this example, the condition is set to **InternetAvailable** so that maintenance runs when the Internet is available (or when it becomes available). For a list of possible background task conditions, see [SystemConditionType](#).

The following code adds a condition to the maintenance task builder:

C#

```
SystemCondition exampleCondition = new  
SystemCondition(SystemConditionType.InternetAvailable);
```

Register the background task

- Register the background task by calling your background task registration function. For more information on registering background tasks, see [Register a background task](#).

The following code registers the maintenance task. Note that it assumes your background task runs in a separate process from your app because it specifies `entryPoint`. If your background task runs in the same process as your app, you do not specify `entryPoint`.

C#

```
string entryPoint = "Tasks.ExampleBackgroundTaskClass";  
string taskName   = "Maintenance background task example";  
  
BackgroundTaskRegistration task = RegisterBackgroundTask(entryPoint,  
taskName, taskTrigger, exampleCondition);
```

ⓘ Note

For all device families except desktop, if the device becomes low on memory, background tasks may be terminated. If an out of memory exception is not surfaced, or the app does not handle it, then the background task will be terminated without warning and without raising the `OnCanceled` event. This helps

to ensure the user experience of the app in the foreground. Your background task should be designed to handle this scenario.

ⓘ Note

Universal Windows Platform apps must call **RequestAccessAsync** before registering any of the background trigger types.

To ensure that your Universal Windows app continues to run properly after you release an update to your app, you must call **RemoveAccess** and then call **RequestAccessAsync** when your app launches after being updated. For more information, see [Guidelines for background tasks](#).

ⓘ Note

Background task registration parameters are validated at the time of registration. An error is returned if any of the registration parameters are invalid. Ensure that your app gracefully handles scenarios where background task registration fails - if instead your app depends on having a valid registration object after attempting to register a task, it may crash.

Related topics

- [Create and register an in-process background task.](#)
- [Create and register an out-of-process background task](#)
- [Declare background tasks in the application manifest](#)
- [Handle a cancelled background task](#)
- [Monitor background task progress and completion](#)
- [Register a background task](#)
- [Respond to system events with background tasks](#)
- [Set conditions for running a background task](#)
- [Update a live tile from a background task](#)
- [Run a background task on a timer](#)
- [Guidelines for background tasks](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Connected apps and devices (Project Rome)

Article • 04/14/2023

This section explains how to connect apps across devices and platforms using **Project Rome**. To learn how to implement Project Rome in a cross-platform scenario, visit the [main docs page for Project Rome](#).

Most users have multiple devices and often begin an activity on one device and finish it on another. To accommodate this, apps need to span devices and platforms. Project Rome allows you to discover remote devices, launch an app on a remote device, and communicate with an app service on a remote device.

The [Remote Systems APIs](#) introduced in Windows 10, version 1607 enable you to write apps that allow users to start a task on one device and finish it on another. The task remains the central focus, and users can do their work on the device that is most convenient. For example, a user might be listening to the radio on their phone in the car, but when they get home they may want to transfer playback to their Xbox One which is hooked up to the home stereo system.

You can also use Project Rome for companion devices or remote control scenarios. Use the app service messaging APIs to create an app channel between two devices to send and receive custom messages. For example, you can write an app for your phone that controls playback on your TV, or a companion app that provides information about the characters on a TV show you are watching through another app.

Devices can be connected proximally through Bluetooth and wireless, or remotely through the cloud; they are linked by the Microsoft account (MSA) of the person using them.

See the [Remote Systems UWP sample](#) [↗](#) for examples of how to discover remote system, launch an app on a remote system, and use app services to send messages between apps running on two systems.

For more information on Project Rome in general, including resources for cross-platform integration, go to aka.ms/project-rome.

Topic	Description
Launch an app on a remote device	Learn how to launch an app on a remote device. This topic covers the simplest use case and preliminary setup.

Topic	Description
Discover remote devices	Learn how to discover devices that you can connect to.
Communicate with a remote app service	Learn how to interact with an app on a remote device.
Connect devices through remote sessions	Create shared experiences across multiple devices by joining them in a remote session.
Continue user activity, even across devices	Help users resume what they were doing in your app, even across multiple devices.
User Activities best practices	Learn the recommended practices for creating and updating User Activities.

Discover remote devices

Article • 10/20/2022

Your app can use the wireless network, Bluetooth, and cloud connection to discover Windows devices that are signed on with the same Microsoft account as the discovering device. The remote devices do not need to have any special software installed in order to be discoverable.

ⓘ Note

This guide assumes you have already been granted access to the Remote Systems feature by following the steps in [Launch a remote app](#).

Filter the set of discoverable devices

You can narrow the set of discoverable devices by using a [RemoteSystemWatcher](#) with filters. Filters can detect the discovery type (proximal vs. local network vs. cloud connection), device type (desktop, mobile device, Xbox, Hub, and Holographic), and availability status (the status of a device's availability to use Remote System features).

Filter objects must be constructed before or while the **RemoteSystemWatcher** object is initialized, because they are passed as a parameter into its constructor. The following code creates a filter of each type available and then adds them to a list.

ⓘ Note

The code in these examples requires that you have a `using Windows.System.RemoteSystems` statement in your file.

C#

```
private List<IRemoteSystemFilter> makeFilterList()
{
    // construct an empty list
    List<IRemoteSystemFilter> localListOfFilters = new
    List<IRemoteSystemFilter>();

    // construct a discovery type filter that only allows "proximal"
    connections:
    RemoteSystemDiscoveryTypeFilter discoveryFilter = new
    RemoteSystemDiscoveryTypeFilter(RemoteSystemDiscoveryType.Proximal);
```

```

        // construct a device type filter that only allows desktop and mobile
        devices:
        // For this kind of filter, we must first create an IEnumerable of strings
        representing the device types to allow.
        // These strings are stored as static read-only properties of the
        RemoteSystemKinds class.
        List<String> listOfTypes = new List<String>();
        listOfTypes.Add(RemoteSystemKinds.Desktop);
        listOfTypes.Add(RemoteSystemKinds.Phone);

        // Put the list of device types into the constructor of the filter
        RemoteSystemKindFilter kindFilter = new
        RemoteSystemKindFilter(listOfTypes);

        // construct an availability status filter that only allows devices
        marked as available:
        RemoteSystemStatusTypeFilter statusFilter = new
        RemoteSystemStatusTypeFilter(RemoteSystemStatusType.Available);

        // add the 3 filters to the listL
        localListOfFilters.Add(discoveryFilter);
        localListOfFilters.Add(kindFilter);
        localListOfFilters.Add(statusFilter);

        // return the list
        return localListOfFilters;
    }

```

ⓘ Note

The "proximal" filter value does not guarantee the degree of physical proximity. For scenarios that require reliable physical proximity, use the value **RemoteSystemDiscoveryType.SpatiallyProximal** in your filter. Currently, this filter only allows devices that are discovered by Bluetooth. As new discovery mechanisms and protocols which guarantee physical proximity are supported, they will be included here as well.

There is also a property in the **RemoteSystem** class that indicates whether a discovered device is in fact within physical proximity:

RemoteSystem.IsAvailableBySpatialProximity.

ⓘ Note

If you intend to discover devices over a local network (determined by your discovery type filter selection), your network needs to be using a "private" or

"domain" profile. Your device will not discover other devices over a "public" network.

Once a list of [IRemoteSystemFilter](#) objects is created, it can be passed into the constructor of a [RemoteSystemWatcher](#).

C#

```
// store filter list
List<IRemoteSystemFilter> listOfFilters = makeFilterList();

// construct watcher with the list
m_remoteSystemWatcher = RemoteSystem.CreateWatcher(listOfFilters);
```

When this watcher's [Start](#) method is called, it will raise the [RemoteSystemAdded](#) event only if a device is detected that meets all of the following criteria:

- It is discoverable by proximal connection
- It is a desktop or phone
- It is classified as available

From there, the procedure for handling events, retrieving [RemoteSystem](#) objects, and connecting to remote devices is exactly the same as in [Launch a remote app](#). In short, the [RemoteSystem](#) objects are stored as properties of [RemoteSystemAddedEventArgs](#) objects, which are passed in with each [RemoteSystemAdded](#) event.

Discover devices by address input

Some devices may not be associated with a user or discoverable with a scan, but they can still be reached if the discovering app uses a direct address. The [HostName](#) class is used to represent the address of a remote device. This is often stored in the form of an IP address, but several other formats are allowed (see the [HostName constructor](#) for details).

A [RemoteSystem](#) object is retrieved if a valid [HostName](#) object is provided. If the address data is invalid, a `null` object reference is returned.

C#

```
private async Task<RemoteSystem> getDeviceByAddressAsync(string IPaddress)
{
    // construct a HostName object
    Windows.Networking.HostName deviceHost = new
Windows.Networking.HostName(IPaddress);
```

```

    // create a RemoteSystem object with the HostName
    RemoteSystem remotesys = await
RemoteSystem.FindByHostNameAsync(deviceHost);

    return remotesys;
}

```

Querying a capability on a remote system

Although separate from discovery filtering, querying device capabilities can be an important part of the discovery process. Using the [RemoteSystem.GetCapabilitySupportedAsync](#) method, you can query discovered remote systems for support of certain capabilities such as remote session connectivity or spatial entity (holographic) sharing. See the [KnownRemoteSystemCapabilities](#) class for the list of queryable capabilities.

C#

```

// Check to see if the given remote system can accept LaunchUri requests
bool isRemoteSystemLaunchUriCapable =
remoteSystem.GetCapabilitySupportedAsync(KnownRemoteSystemCapabilities.Launc
hUri);

```

Cross-user discovery

Developers can specify the discovery of *all* devices in proximity to the client device, not just devices registered to the same user. This is implemented through a special [IRemoteSystemFilter](#), [RemoteSystemAuthorizationKindFilter](#). It is implemented like the other filter types:

C#

```

// Construct a user type filter that includes anonymous devices
RemoteSystemAuthorizationKindFilter authorizationKindFilter = new
RemoteSystemAuthorizationKindFilter(RemoteSystemAuthorizationKind.Anonymous)
;
// then add this filter to the RemoteSystemWatcher

```

- A [RemoteSystemAuthorizationKind](#) value of **Anonymous** will allow the discovery of all proximal devices, even those from non-trusted users.
- A value of **SameUser** filters the discovery to only devices registered to the same user as the client device. This is the default behavior.

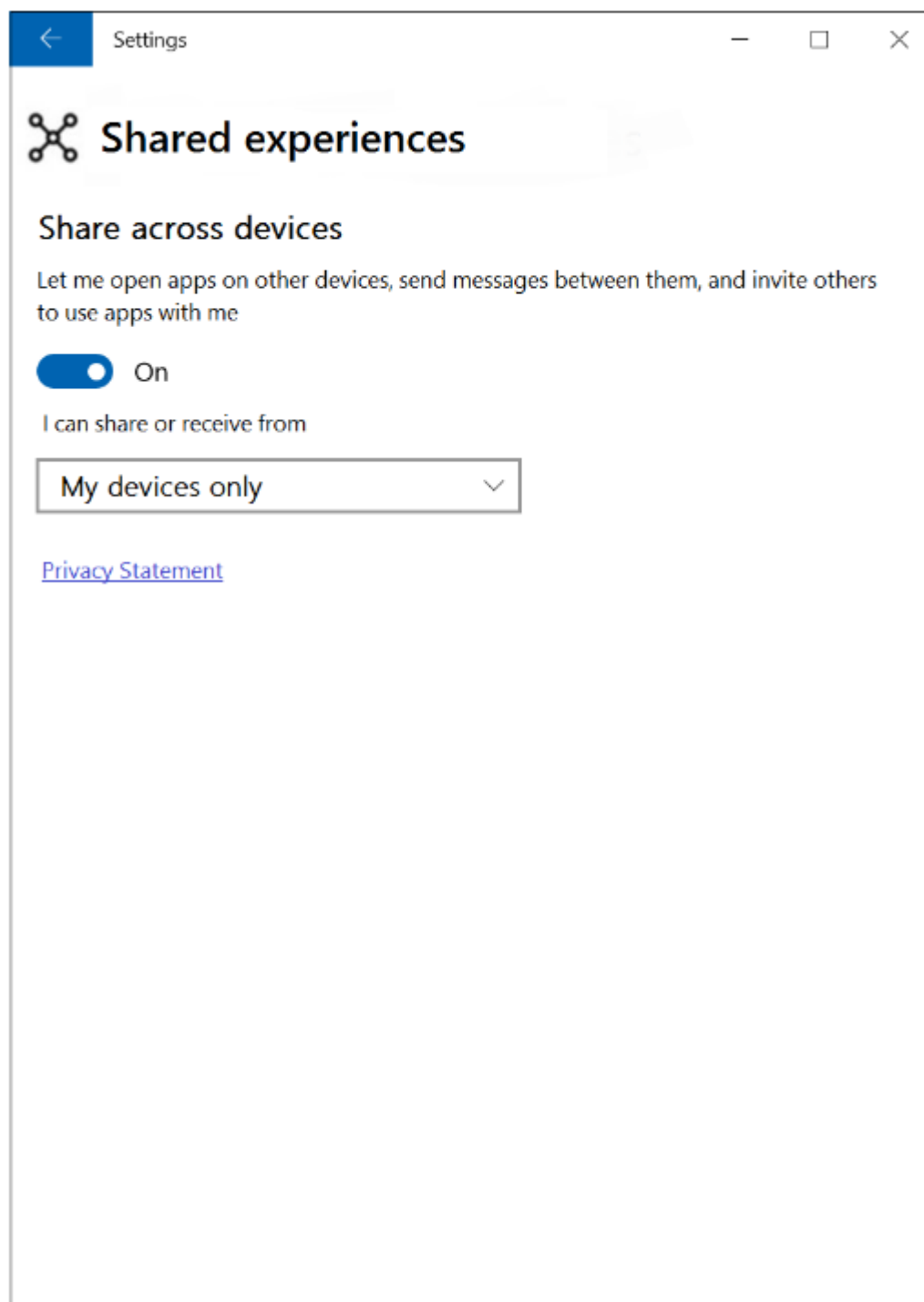
Checking the Cross-User Sharing settings

In addition to the above filter being specified in your discovery app, the client device itself must also be configured to allow shared experiences from devices signed in with other users. This is a system setting that can be queried with a static method in the **RemoteSystem** class:

C#

```
if
(!RemoteSystem.IsAuthorizationKindEnabled(RemoteSystemAuthorizationKind.Anon
ymous)) {
    // The system is not authorized to connect to cross-user devices.
    // Inform the user that they can discover more devices if they
    // update the setting to "Anonymous".
}
```

To change this setting, the user must open the **Settings** app. In the **System > Shared experiences > Share across devices** menu, there is a drop-down box where the user can specify which devices their system can share with.



Related topics

- [Connected apps and devices \(Project Rome\)](#)
- [Launch a remote app](#)
- [Remote Systems API reference](#)
- [Remote Systems sample](#) 

Launch an app on a remote device

Article • 10/20/2022

This article explains how to launch a Windows app on a remote device.

Starting in Windows 10, version 1607, a UWP app can launch a UWP app or Windows desktop application remotely on another device that is also running Windows 10, version 1607 or later, provided that both devices are signed on with the same Microsoft Account (MSA). This is the simplest use case of Project Rome.

The remote launch feature enables task-oriented user experiences; a user can start a task on one device and finish it on another. For example, if the user is listening to music on their phone in their car, they could then hand playback functionality over to their Xbox One when they arrive at home. Remote launch allows apps to pass contextual data to the remote app being launched, in order to pick up where the task was left off.

Preliminary setup

Add the `remoteSystem` capability

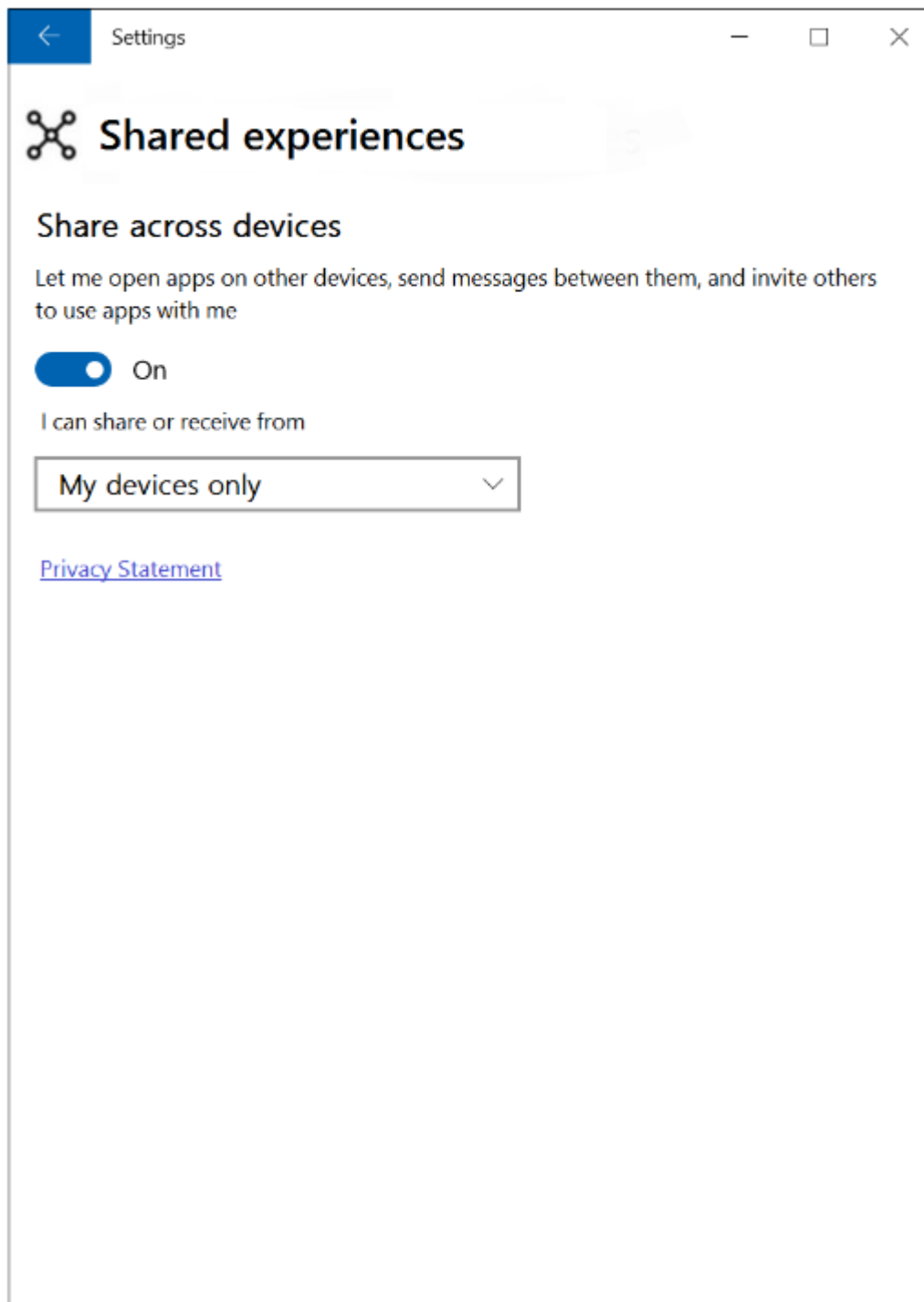
In order for your app to launch an app on a remote device, you must add the `remoteSystem` capability to your app package manifest. You can use the package manifest designer to add it by selecting **Remote System** on the **Capabilities** tab, or you can manually add the following line to your project's *Package.appxmanifest* file.

XML

```
<Capabilities>
  <uap3:Capability Name="remoteSystem"/>
</Capabilities>
```

Enable cross-device sharing

Additionally, the client device must be set to allow cross-device sharing. This setting, which is accessed in **Settings: System > Shared experiences > Share across devices**, is enabled by default.



Find a remote device

You must first find the device that you want to connect with. [Discover remote devices](#) discusses how to do this in detail. We'll use a simple approach here that forgoes filtering by device or connectivity type. We will create a remote system watcher that looks for remote devices, and write handlers for the events that are raised when devices are discovered or removed. This will provide us with a collection of remote devices.

The code in these examples requires that you have a `using Windows.System.RemoteSystems` statement in your class file(s).

C#

```
private async Task BuildDeviceList()
{
    RemoteSystemAccessStatus accessStatus = await
RemoteSystem.RequestAccessAsync();

    if (accessStatus == RemoteSystemAccessStatus.Allowed)
    {
        m_remoteSystemWatcher = RemoteSystem.CreateWatcher();

        // Subscribing to the event raised when a new remote system is found
        by the watcher.
        m_remoteSystemWatcher.RemoteSystemAdded +=
RemoteSystemWatcher_RemoteSystemAdded;

        // Subscribing to the event raised when a previously found remote
        system is no longer available.
        m_remoteSystemWatcher.RemoteSystemRemoved +=
RemoteSystemWatcher_RemoteSystemRemoved;

        m_remoteSystemWatcher.Start();
    }
}
```

The first thing you must do before making a remote launch is call `RemoteSystem.RequestAccessAsync()`. Check the return value to make sure your app is allowed to access remote devices. One reason this check could fail is if you haven't added the `remoteSystem` capability to your app.

The system watcher event handlers are called when a device that we can connect with is discovered or is no longer available. We will use these event handlers to keep an updated list of devices that we can connect to.

C#

```
private void RemoteSystemWatcher_RemoteSystemRemoved(
    RemoteSystemWatcher sender, RemoteSystemRemovedEventArgs args)
{
    if ( m_deviceMap.ContainsKey(args.RemoteSystemId))
    {
        m_deviceList.Remove(m_deviceMap[args.RemoteSystemId]);
        m_deviceMap.Remove(args.RemoteSystemId);
    }
}

private void RemoteSystemWatcher_RemoteSystemAdded(
    RemoteSystemWatcher sender, RemoteSystemAddedEventArgs args)
{
    m_deviceList.Add(args.RemoteSystem);
}
```

```
m_deviceMap.Add(args.RemoteSystem.Id, args.RemoteSystem);  
}
```

We will track the devices by remote system ID using a **Dictionary**. An **ObservableCollection** is used to hold the list of devices that we can enumerate. An **ObservableCollection** also makes it easy to bind the list of devices to UI, though we won't do that in this example.

C#

```
private RemoteSystemWatcher m_remoteSystemWatcher;  
private ObservableCollection<RemoteSystem> m_deviceList = new  
ObservableCollection<RemoteSystem>();  
private Dictionary<string, RemoteSystem> m_deviceMap = new  
Dictionary<string, RemoteSystem>();
```

Add a call to `BuildDeviceList()` in your app startup code before you attempt to launch a remote app.

Launch an app on a remote device

Launch an app remotely by passing the device you wish to connect with to the [RemoteLauncher.LaunchUriAsync](#) API. There are three overloads for this method. The simplest, which this example demonstrates, specifies the URI that will activate the app on the remote device. In this example the URI opens the Maps app on the remote machine with a 3D view of the Space Needle.

Other [RemoteLauncher.LaunchUriAsync](#) overloads allow you to specify options such as the URI of the web site to view if no appropriate app can be launched on the remote device, and an optional list of package family names that could be used to launch the URI on the remote device. You can also provide data in the form of key/value pairs. You might pass data to the app you are activating to provide context to the remote app, such as the name of the song to play and the current playback location when you hand off playback from one device to another.

In practical scenarios, you might provide UI to select the device you want to target. But to simplify this example, we'll just use the first remote device on the list.

C#

```
if ( m_deviceList.Count > 0)  
{  
    RemoteSystem SelectedDevice = m_deviceList[0];  
    RemoteLaunchUriStatus launchUriStatus =
```

```
await RemoteLauncher.LaunchUriAsync(  
    new RemoteSystemConnectionRequest(SelectedDevice),  
    new Uri("bingmaps:?  
cp=47.6204~-122.3491&sty=3d&rad=200&pit=75&hdg=165"));  
}
```

The [RemoteLaunchUriStatus](#) object that is returned from `RemoteLauncher.LaunchUriAsync()` provides information about whether the remote launch succeeded, and if not, the reason why.

Related topics

[Remote Systems API reference](#)

[Connected apps and devices \(Project Rome\) overview](#)

[Discover remote devices](#)

[Remote Systems sample](#) [↗](#) demonstrates how to discover a remote system, launch an app on a remote system, and use app services to send messages between apps running on two systems.

Communicate with a remote app service

Article • 10/20/2022

In addition to launching an app on a remote device using a URI, you can run and communicate with *app services* on remote devices as well. Any Windows-based device can be used as either the client or host device. This gives you an almost limitless number of ways to interact with connected devices without needing to bring an app to the foreground.

Set up the app service on the host device

In order to run an app service on a remote device, you must already have a provider of that app service installed on that device. This guide will use CSharp version of the [Random Number Generator app service sample](#), which is available on the [Windows universal samples repo](#). For instructions on how to write your own app service, see [Create and consume an app service](#).

Whether you are using an already-made app service or writing your own, you will need to make a few edits in order to make the service compatible with remote systems. In Visual Studio, go to the app service provider's project (called "AppServiceProvider" in the sample) and select its *Package.appxmanifest* file. Right-click and select **View Code** to view the full contents of the file. Create an **Extensions** element inside of the main **Application** element (or find it if it already exists). Then create an **Extension** to define the project as an app service and reference its parent project.

XML

```
...
<Extensions>
  <uap:Extension Category="windows.appService"
    EntryPoint="RandomNumberService.RandomNumberGeneratorTask">
    <uap3:AppService Name="com.microsoft.randomnumbergenerator"/>
  </uap:Extension>
</Extensions>
...
```

Next to the **AppService** element, add the **SupportsRemoteSystems** attribute:

XML

```
...
<uap3:AppService Name="com.microsoft.randomnumbergenerator"
  SupportsRemoteSystems="true"/>
```

```
SupportsRemoteSystems="true"/>
...
```

In order to use elements in this **uap3** namespace, you must add the namespace definition at the top of the manifest file if it isn't already there.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<Package
  xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:mp="http://schemas.microsoft.com/appx/2014/phone/manifest"
  xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"
  xmlns:uap3="http://schemas.microsoft.com/appx/manifest/uap/windows10/3">
  ...
</Package>
```

Then build your app service provider project and deploy it to the host device(s).

Target the app service from the client device

The device from which the remote app service is to be called needs an app with Remote Systems functionality. This can be added into the same app that provides the app service on the host device (in which case you would install the same app on both devices), or implemented in a completely different app.

The following **using** statements are needed for the code in this section to run as-is:

C#

```
using Windows.ApplicationModel.AppService;
using Windows.System.RemoteSystems;
```

You must first instantiate an **AppServiceConnection** object, just as if you were to call an app service locally. This process is covered in more detail in [Create and consume an app service](#). In this example, the app service to target is the Random Number Generator service.

ⓘ Note

It is assumed that a **RemoteSystem** object has already been acquired by some means within the code that would call the following method. See [Launch a remote app](#) for instructions on how to set this up.

C#

```
// This method returns an open connection to a particular app service on a
remote system.
// param "remotesys" is a RemoteSystem object representing the device to
connect to.
private async void openRemoteConnectionAsync(RemoteSystem remotesys)
{
    // Set up a new app service connection. The app service name and package
    family name that
    // are used here correspond to the AppServices UWP sample.
    AppServiceConnection connection = new AppServiceConnection
    {
        AppServiceName = "com.microsoft.randomnumbergenerator",
        PackageFamilyName =
        "Microsoft.SDKSamples.AppServicesProvider.CS_8wekyb3d8bbwe"
    };
}
```

Next, a [RemoteSystemConnectionRequest](#) object is created for the intended remote device. It is then used to open the **AppServiceConnection** to that device. Note that in the example below, error handling and reporting is greatly simplified for brevity.

C#

```
// a valid RemoteSystem object is needed before going any further
if (remotesys == null)
{
    return;
}

// Create a remote system connection request for the given remote device
RemoteSystemConnectionRequest connectionRequest = new
RemoteSystemConnectionRequest(remotesys);

// "open" the AppServiceConnection using the remote request
AppServiceConnectionStatus status = await
connection.OpenRemoteAsync(connectionRequest);

// only continue if the connection opened successfully
if (status != AppServiceConnectionStatus.Success)
{
    return;
}
```

At this point, you should have an open connection to an app service on a remote machine.

Exchange service-specific messages over the remote connection

From here, you can send and receive messages to and from the service in the form of [ValueSet](#) objects (for more information, see [Create and consume an app service](#)). The Random number generator service takes two integers with the keys "minvalue" and "maxvalue" as inputs, randomly selects an integer within their range, and returns it to the calling process with the key "Result".

C#

```
// create the command input
ValueSet inputs = new ValueSet();

// min_value and max_value vars are obtained somewhere else in the
program
inputs.Add("minvalue", min_value);
inputs.Add("maxvalue", max_value);

// send input and receive output in a variable
AppServiceResponse response = await connection.SendMessageAsync(inputs);

string result = "";
// check that the service successfully received and processed the
message
if (response.Status == AppServiceResponseStatus.Success)
{
    // Get the data that the service returned:
    result = response.Message["Result"] as string;
}
}
```

Now you have connected to an app service on a targeted host device, run an operation on that device, and received data to your client device in response.

Related topics

[Connected apps and devices \(Project Rome\) overview](#)

[Launch a remote app](#)

[Create and consume an app service](#)

[Remote Systems API reference](#)

[Remote Systems sample](#) ↗

Connect devices through remote sessions

Article • 10/20/2022

The Remote Sessions feature allows an app to connect to other devices through a session, either for explicit app messaging or for brokered exchange of system-managed data, such as the [SpatialEntityStore](#) for holographic sharing between Windows Holographic devices.

Remote sessions can be created by any Windows device, and any Windows device can request to join (although sessions can have invite-only visibility), including devices signed in by other users. This guide provides basic sample code for all of the major scenarios that make use of remote sessions. This code can be incorporated into an existing app project and modified as necessary. For an end-to-end implementation, see the [Quiz Game sample app](#) [↗](#)).

Preliminary setup

Add the `remoteSystem` capability

In order for your app to launch an app on a remote device, you must add the `remoteSystem` capability to your app package manifest. You can use the package manifest designer to add it by selecting **Remote System** on the **Capabilities** tab, or you can manually add the following line to your project's *Package.appxmanifest* file.

XML

```
<Capabilities>
  <uap3:Capability Name="remoteSystem"/>
</Capabilities>
```

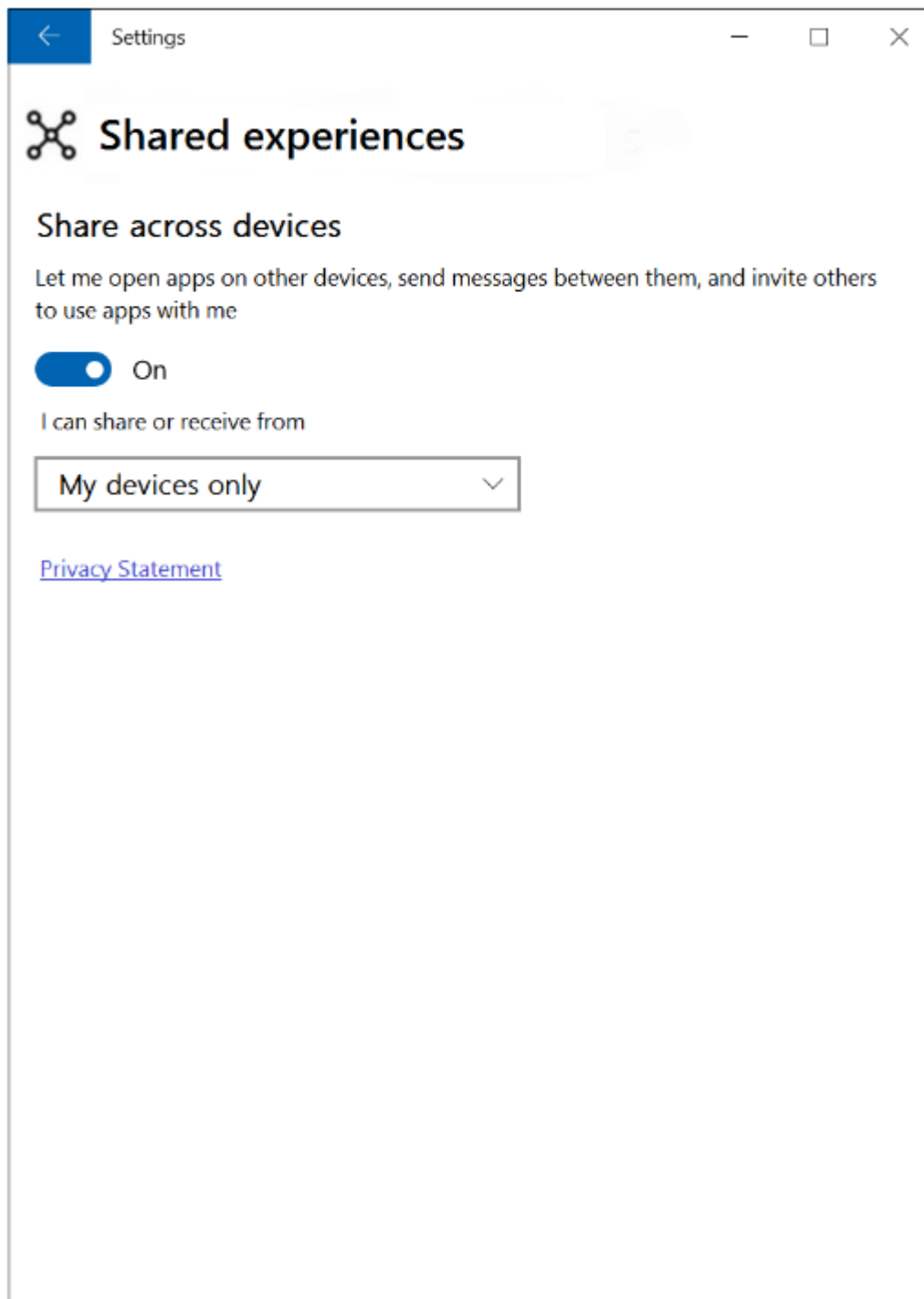
Enable cross-user discovering on the device

Remote Sessions is geared toward connecting multiple different users, so the devices involved will need to have Cross-User Sharing enabled. This is a system setting that can be queried with a static method in the `RemoteSystem` class:

C#

```
if  
(!RemoteSystem.IsAuthorizationKindEnabled(RemoteSystemAuthorizationKind.Anon  
ymous)) {  
    // The system is not authorized to connect to cross-user devices.  
    // Inform the user that they can discover more devices if they  
    // update the setting to "Everyone nearby".  
}
```

To change this setting, the user must open the **Settings** app. In the **System > Shared experiences > Share across devices** menu, there is a drop-down box where the user can specify which devices their system can share with.



Include the necessary namespaces

In order to use all of the code snippets in this guide, you will need the following `using` statements in your class file(s).

C#

```
using System.Runtime.Serialization.Json;
using Windows.Foundation.Collections;
using Windows.System.RemoteSystems;
```

Create a remote session

To create a remote session instance, you must start with a [RemoteSystemSessionController](#) object. Use the following framework to create a new session and handle join requests from other devices.

C#

```
public async void CreateSession() {

    // create a session controller
    RemoteSystemSessionController manager = new
    RemoteSystemSessionController("Bob's Minecraft game");

    // register the following code to handle the JoinRequested event
    manager.JoinRequested += async (sender, args) => {
        // Get the deferral
        var deferral = args.GetDeferral();

        // display the participant (args.JoinRequest.Participant) on UI,
        giving the
        // user an opportunity to respond
        // ...

        // If the user chooses "accept", accept this remote system as a
        participant
        args.JoinRequest.Accept();
    };

    // create and start the session
    RemoteSystemSessionCreationResult createResult = await
    manager.CreateSessionAsync();

    // handle the creation result
    if (createResult.Status == RemoteSystemSessionCreationStatus.Success) {
        // creation was successful, get a reference to the session
        RemoteSystemSession currentSession = createResult.Session;
    }
}
```

```

        // optionally subscribe to the disconnection event
        currentSession.Disconnected += async (sender, args) => {
            // update the UI, using args.Reason
            //...
        };

        // Use session (see later section)
        //...

    } else if (createResult.Status ==
RemoteSystemSessionCreationStatus.SessionLimitsExceeded) {
        // creation failed. Optionally update UI to indicate that there are
        too many sessions in progress
    } else {
        // creation failed for an unknown reason. Optionally update UI
    }
}

```

Make a remote session invite-only

If you wish to keep your remote session from being publicly discoverable, you can make it invite-only. Only the devices that receive an invitation will be able to send join requests.

The procedure is mostly the same as above, but when constructing the [RemoteSystemSessionController](#) instance, you will pass in a configured [RemoteSystemSessionOptions](#) object.

```

C#

// define the session options with the invite-only designation
RemoteSystemSessionOptions sessionOptions = new
RemoteSystemSessionOptions();
sessionOptions.IsInviteOnly = true;

// create the session controller
RemoteSystemSessionController manager = new
RemoteSystemSessionController("Bob's Minecraft game", sessionOptions);

//...

```

To send an invitation, you must have a reference to the receiving remote system (acquired through normal remote system discovery). Simply pass this reference into the session object's [SendInvitationAsync](#) method. All of the participants in a session have a reference to the remote session (see next section), so any participant can send an invitation.

C#

```
// "currentSession" is a reference to a RemoteSystemSession.  
// "guestSystem" is a previously discovered RemoteSystem instance  
currentSession.SendInvitationAsync(guestSystem);
```

Discover and join a remote session

The process of discovering remote sessions is handled by the [RemoteSystemSessionWatcher](#) class and is similar to discovering individual remote systems.

C#

```
public void DiscoverSessions() {  
  
    // create a watcher for remote system sessions  
    RemoteSystemSessionWatcher sessionWatcher =  
    RemoteSystemSession.CreateWatcher();  
  
    // register a handler for the "added" event  
    sessionWatcher.Added += async (sender, args) => {  
  
        // get a reference to the info about the discovered session  
        RemoteSystemSessionInfo sessionInfo = args.SessionInfo;  
  
        // Optionally update the UI with the sessionInfo.DisplayName and  
        // sessionInfo.ControllerDisplayName strings.  
        // Save a reference to this RemoteSystemSessionInfo to use when the  
        // user selects this session from the UI  
        //...  
    };  
  
    // Begin watching  
    sessionWatcher.Start();  
}
```

When a [RemoteSystemSessionInfo](#) instance is obtained, it can be used to issue a join request to the device that controls the corresponding session. An accepted join request will asynchronously return a [RemoteSystemSessionJoinResult](#) object that contains a reference to the joined session.

C#

```
public async void JoinSession(RemoteSystemSessionInfo sessionInfo) {  
  
    // issue a join request and wait for result.  
    RemoteSystemSessionJoinResult joinResult = await
```

```

sessionInfo.JoinAsync();
    if (joinResult.Status == RemoteSystemSessionJoinStatus.Success) {
        // Join request was approved

        // RemoteSystemSession instance "currentSession" was declared at
        class level.
        // Assign the value obtained from the join result.
        currentSession = joinResult.Session;

        // note connection and register to handle disconnection event
        bool isConnected = true;
        currentSession.Disconnected += async (sender, args) => {
            isConnected = false;

            // update the UI with args.Reason value
        };

        if (isConnected) {
            // optionally use the session here (see next section)
            //...
        }
    } else {
        // Join was unsuccessful.
        // Update the UI, using joinResult.Status value to show cause of
        failure.
    }
}

```

A device can be joined to multiple sessions at the same time. For this reason, it may be desirable to separate the joining functionality from the actual interaction with each session. As long as a reference to the [RemoteSystemSession](#) instance is maintained in the app, communication can be attempted over that session.

Share messages and data through a remote session

Receive messages

You can exchange messages and data with other participant devices in the session by using a [RemoteSystemSessionMessageChannel](#) instance, which represents a single session-wide communication channel. As soon as it's initialized, it begins listening for incoming messages.

📌 Note